

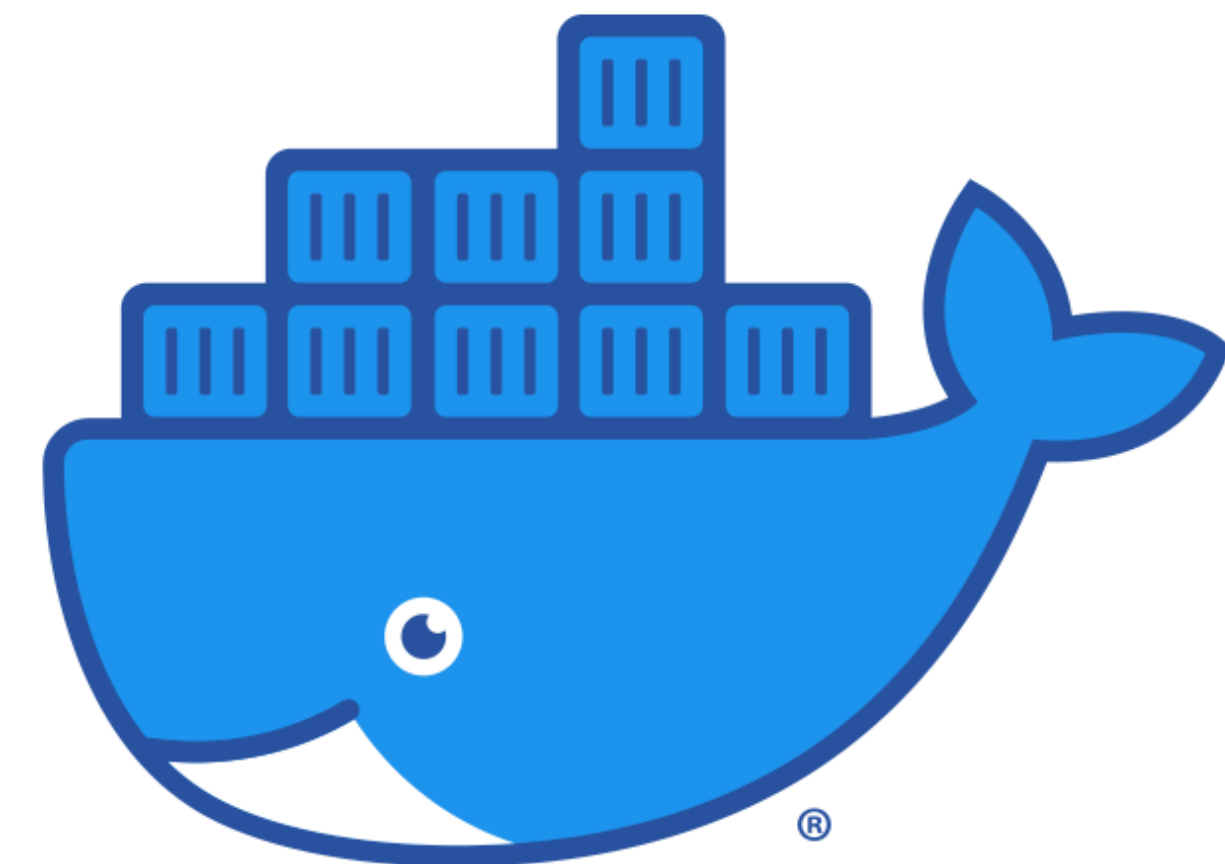
Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Docker

- Docker is a system for running “pretend” (virtual) computers inside a real computer.
- These virtual computers are called “containers”
- <https://docs.docker.com/desktop/mac/install/>
- <https://docs.docker.com/desktop/windows/install/>

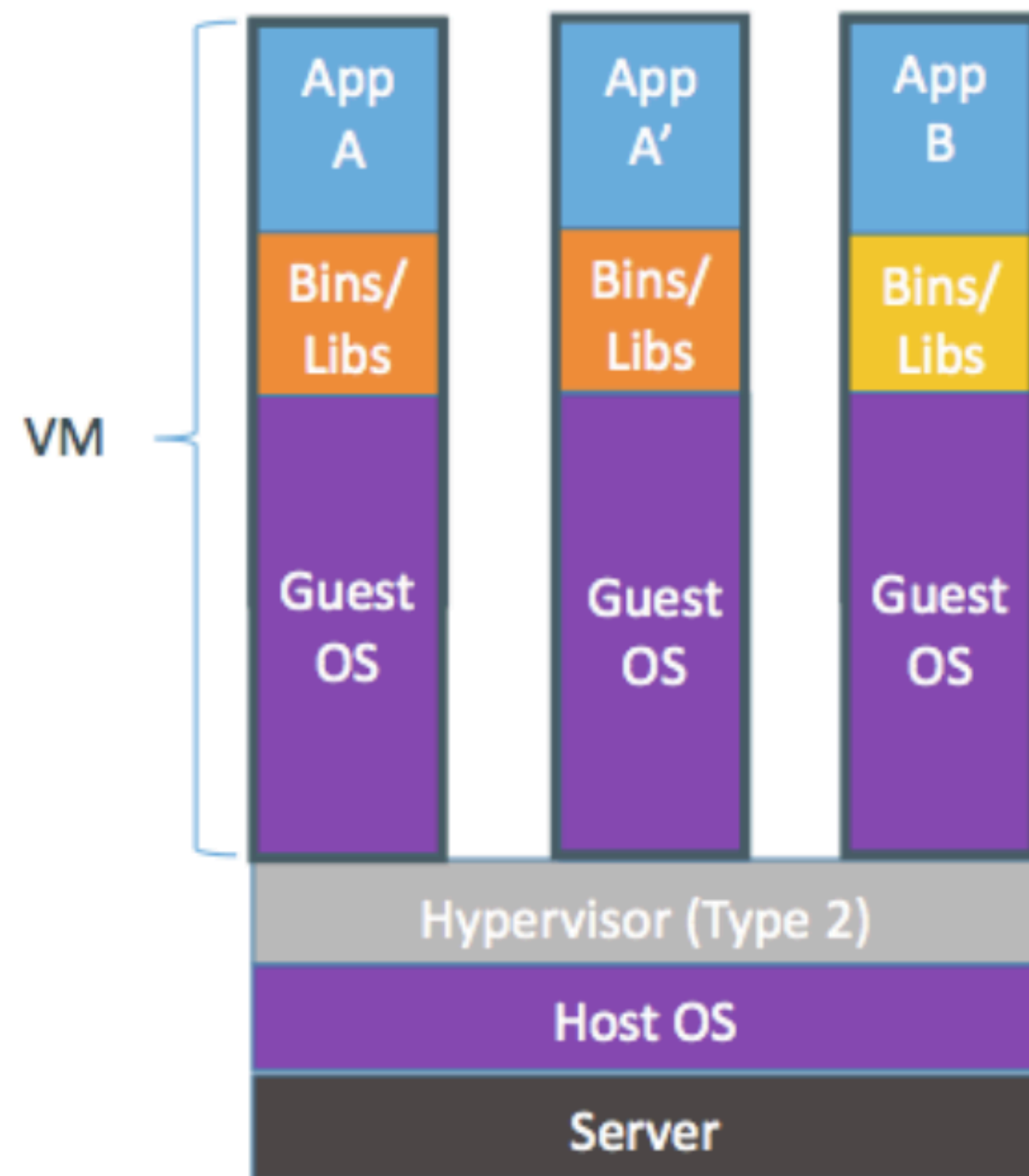


Docker

- Docker containers provide a consistent, contained environment on your local machine
- All students get the same environment and you can't mess up your computer by mistake.
- Docker containers are lightweight, portable, and boot quickly.
- Real world uses of Docker include cloud applications, software testing, and hardware-agnostic software development.

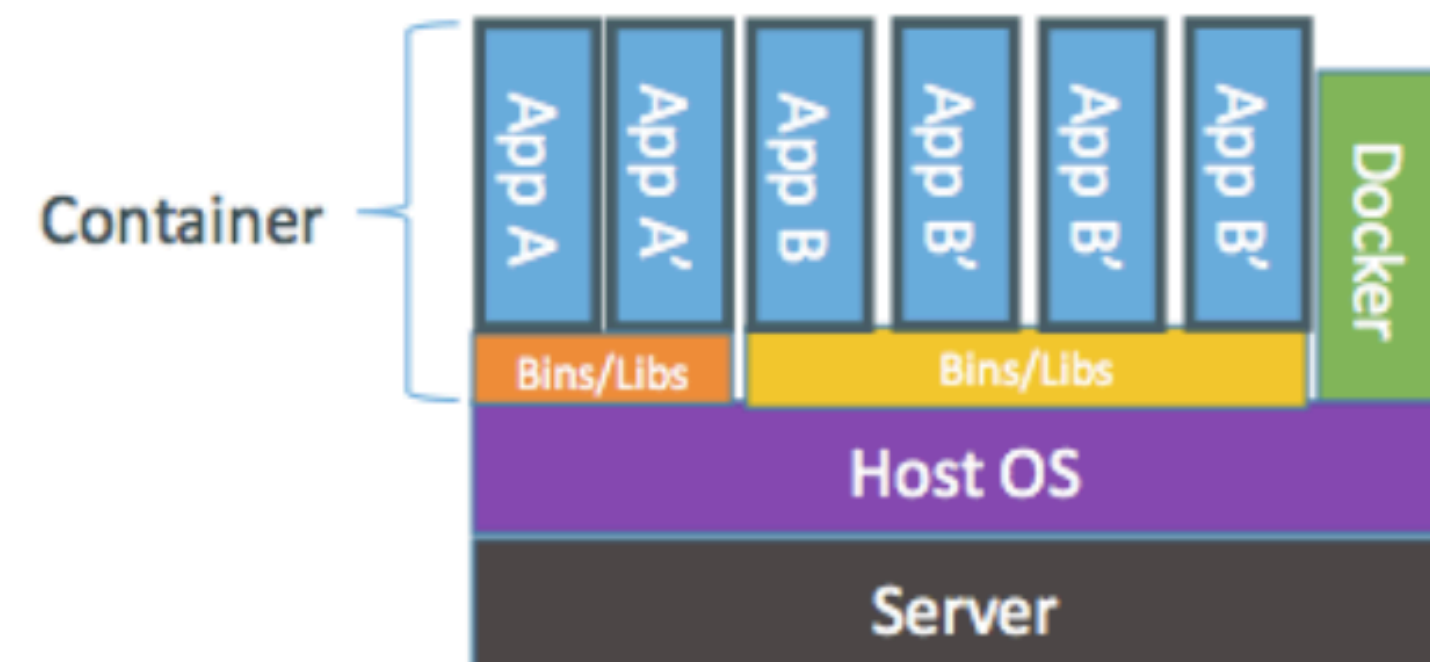
I containers

Sono “versioni leggere” di VM.



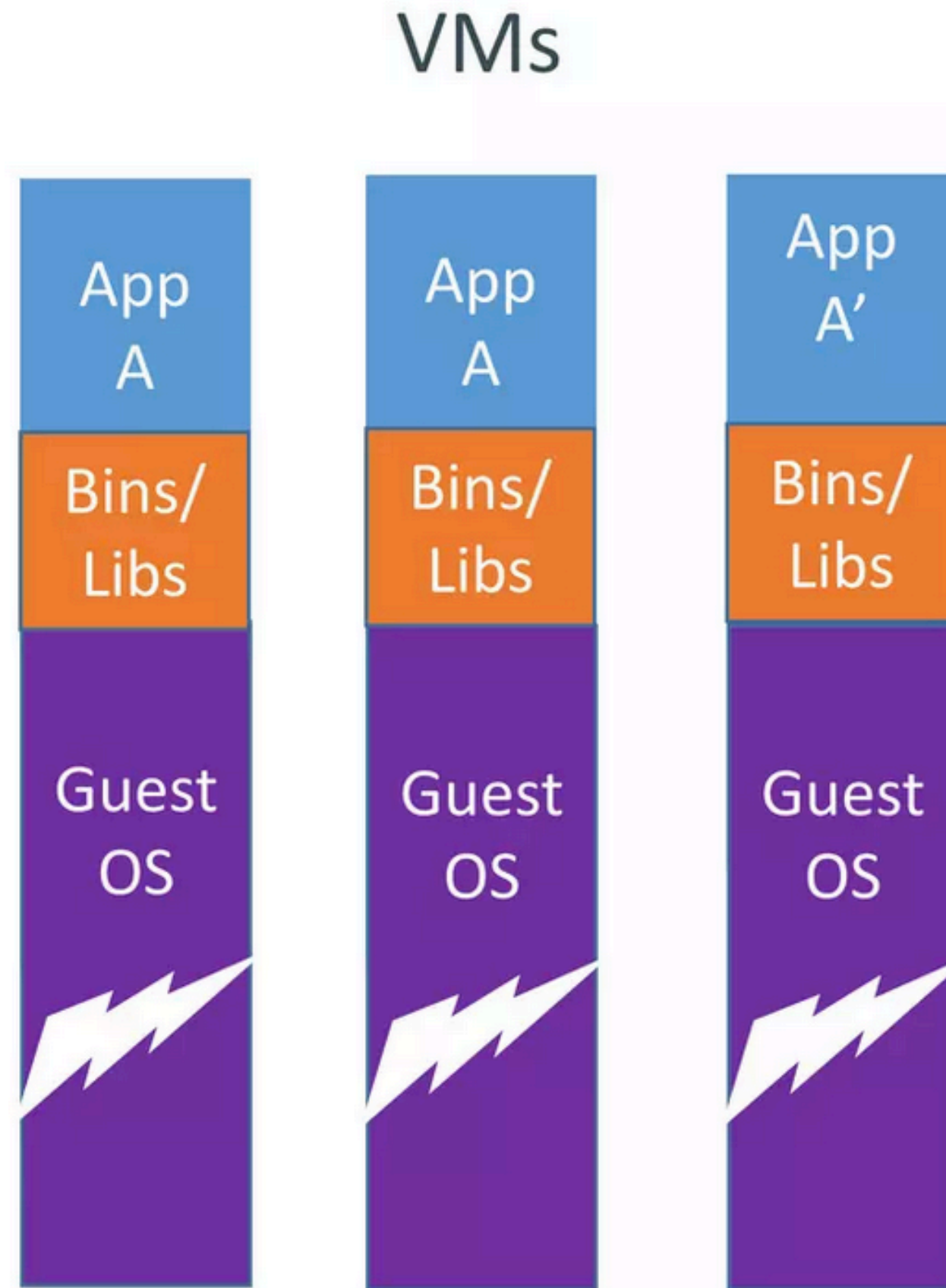
Containers are isolated, but share OS and, where appropriate, bins/libraries

...result is significantly faster deployment, much less overhead, easier migration, faster restart



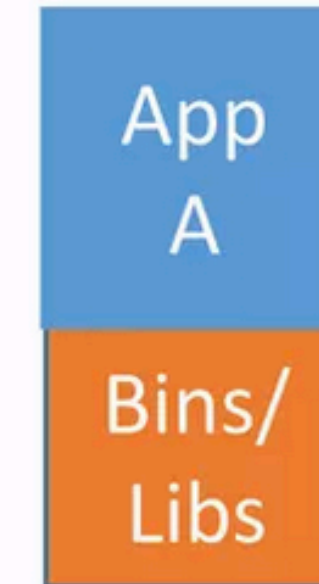
Fonte: Docker (<http://goo.gl/4jh8cX>)

Perchè sono Lightweight?



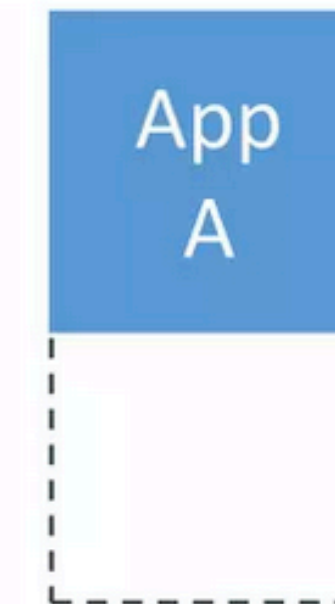
VMs

Every app, every copy of an app, and every slight modification of the app requires a new virtual server

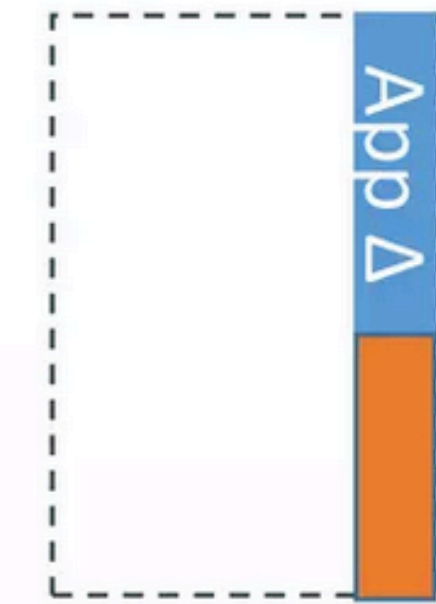


Original App
(No OS to take up space, resources, or require restart)

Containers



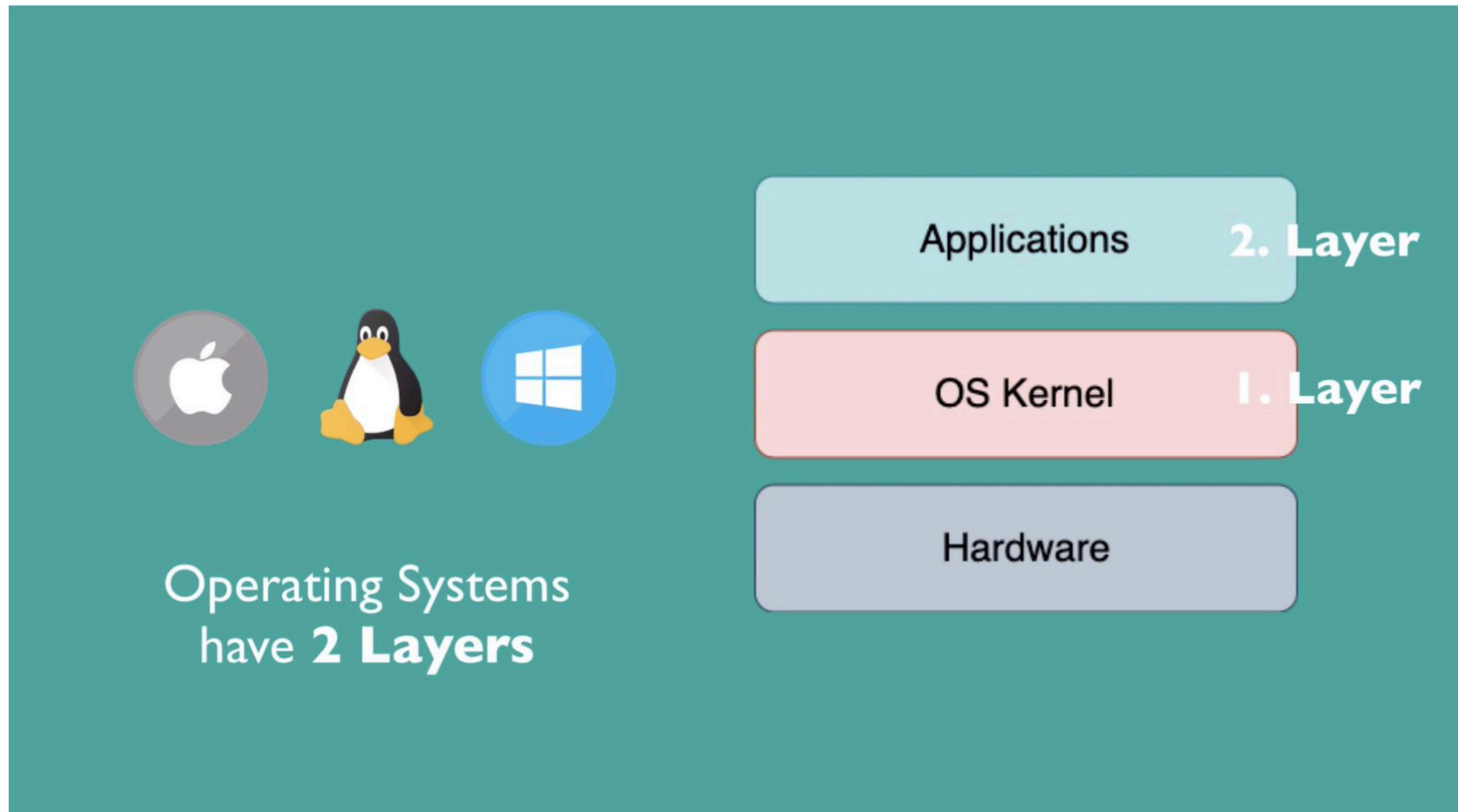
Copy of App
No OS. Can Share bins/libs



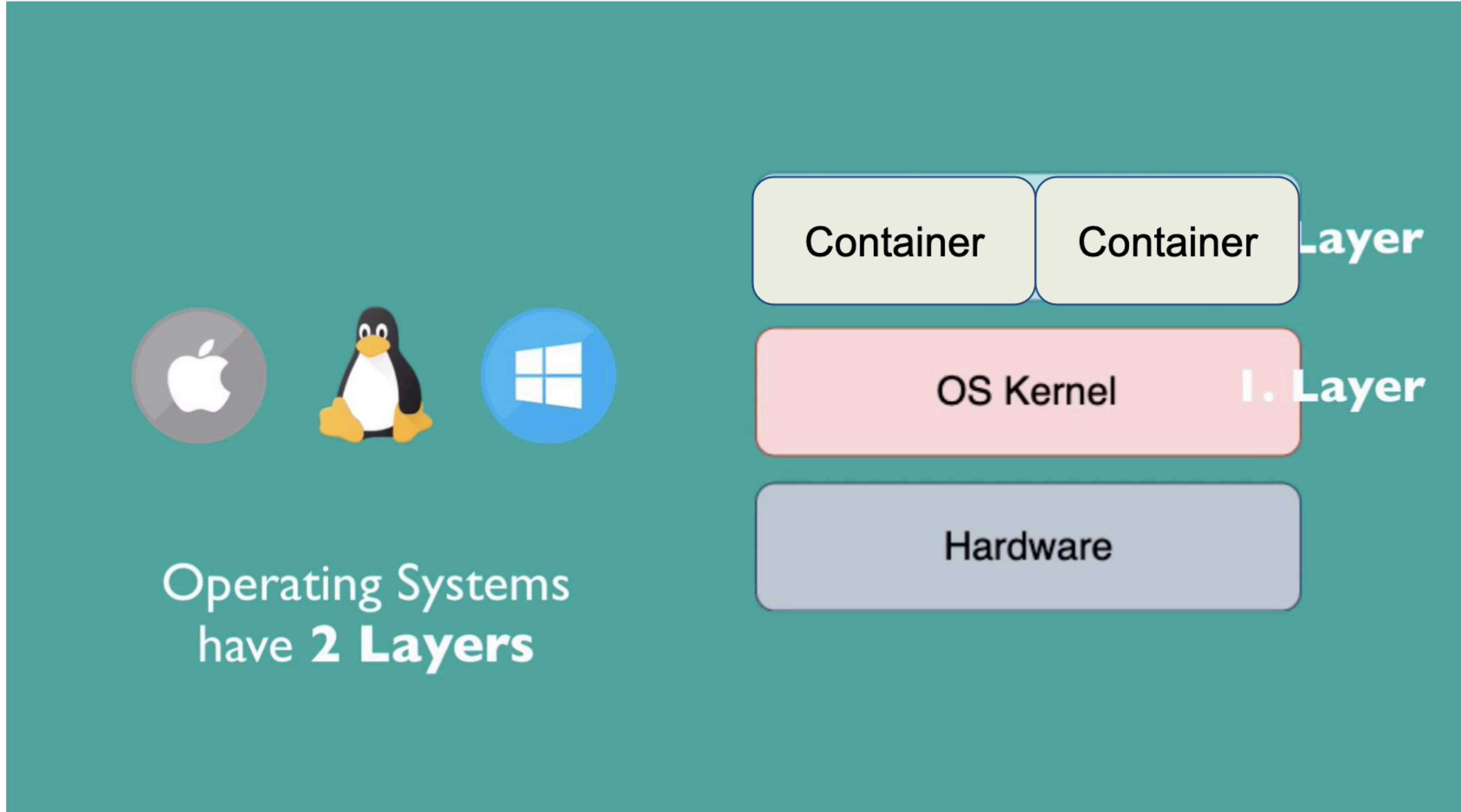
Modified App

Copy on write capabilities allow us to only save the diffs Between container A and container A'

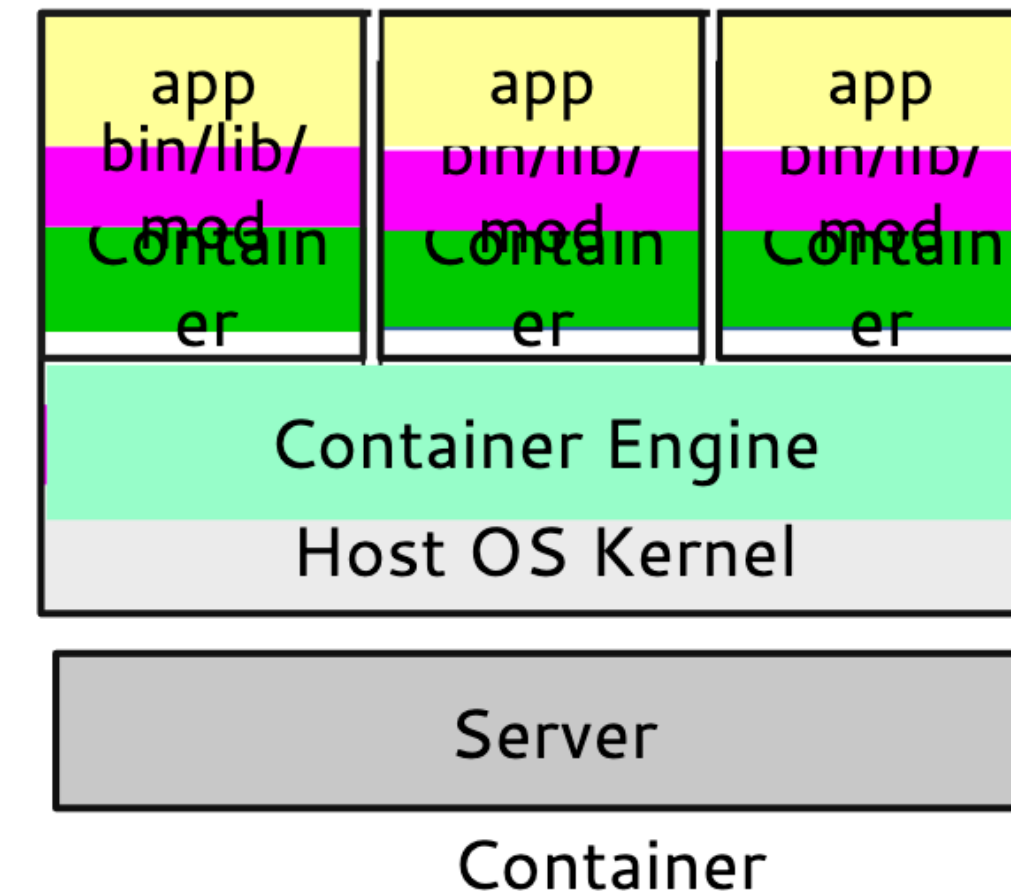
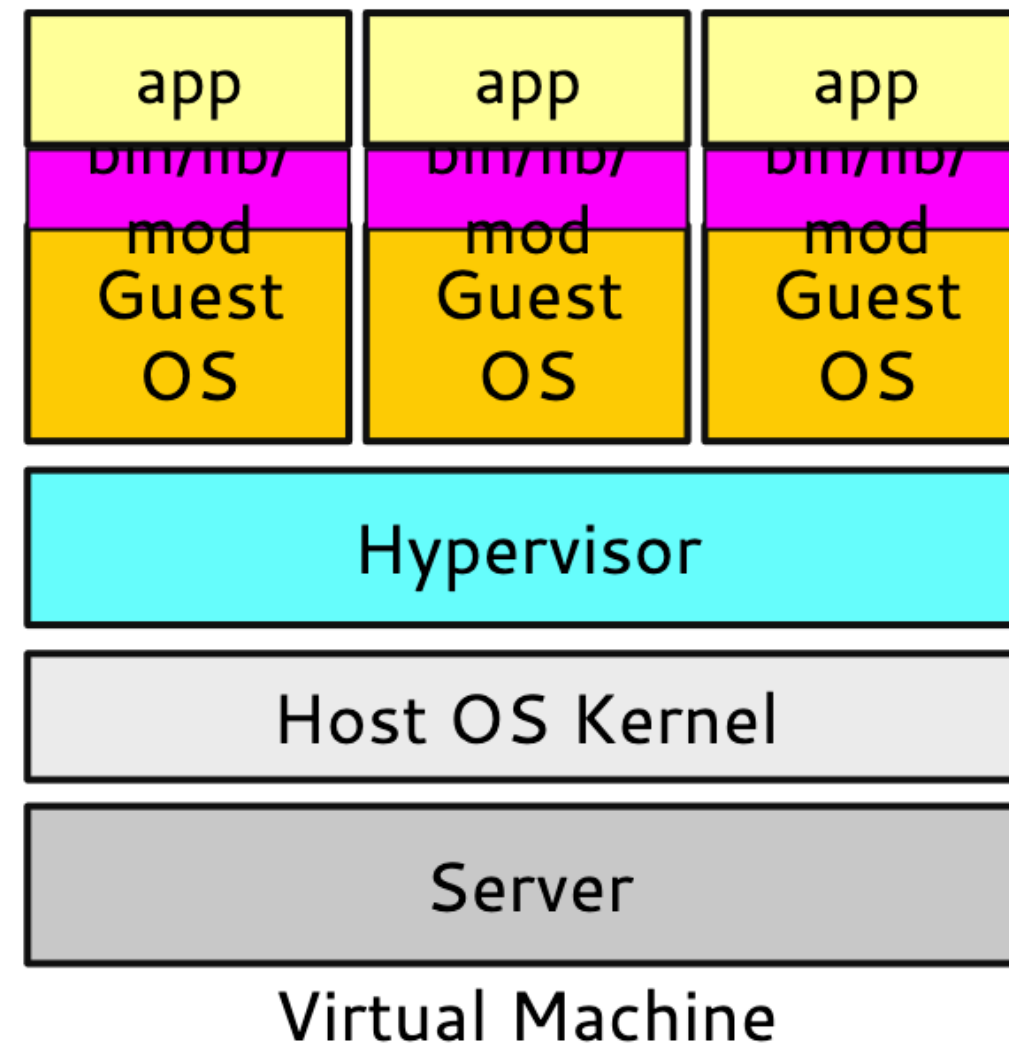
Come funziona un container?



Come funziona un container?



Containers vs VMs



- ★ Virtual Machines are able to run different Operative Systems
- ★ Container Engine: abstraction and isolation level between OS and applications environment. Enables and disables containers. Packages applications and their environment
- ★ Containers virtualize at the OS level, with multiple containers running atop the OS kernel directly. Containers are far more lightweight: they share the OS kernel, start much faster, and use a fraction of the memory compared to booting an entire OS

Containers vs VMs

- Docker containers are typically **stateless** (container storage is erased when the container stops)!
- Docker uses OS-level virtualization, so containers share the host operating system without the need for a hypervisor. This means containers are typically smaller and faster than VMs.
- The Docker platform can run many containers simultaneously and facilitates communication between them
- Online Docker registries allow straightforward deployment of Docker images
- Docker is free for educational use

Come funziona un container?

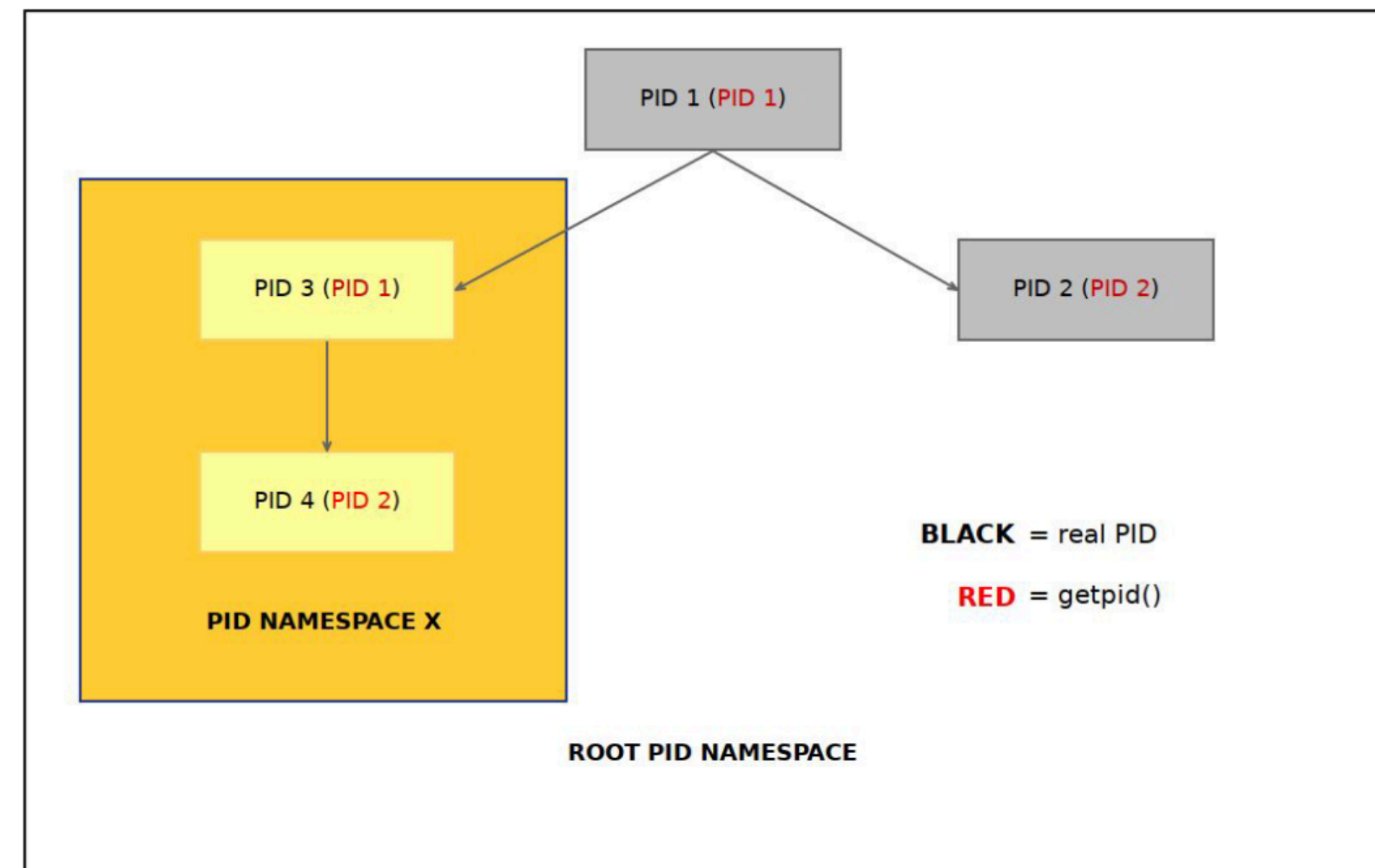
- **Client:** The way you interact with Docker. For us, this will be on the command line.
- **Daemon:** A background service that listens for API requests and manages Docker objects.
- **Docker Desktop:** An easy-to-install application that includes a client (both CLI and GUI), a daemon, and other stuff.
- **Docker registry:** A place where Docker images are stored. We will use Docker Hub, a public registry that is configured in Docker by default.
- **Docker Objects:**
 - *An **image** is a read-only template for a container.*
 - *A **container** is a runnable instance of an image that can be started/stopped/paused etc.*

Come funziona un container?

★ Process Isolation and sandbox

★ **A namespace:** wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of the global resource. Changes to the global resource are visible to other processes that are members of the namespace, but are invisible to other processes.

★ **Cgroups:** limit, police and account the resource usage for a set of processes.



Quali sono i vantaggi?

★ **Isolation** Containers virtualize CPU, memory, storage, and network resources at the OS-level, providing developers with a sandboxed view of the OS logically isolated from other applications. Developers, using containers, are able to create predictable environments isolated from other applications.

★ **Productivity enhancement** Containers can include software dependencies needed by the application (specific versions of programming language runtimes, software libraries) guaranteed to be consistent no matter where the application is deployed. All this translates to productivity: developers and IT operations teams spend less time debugging and diagnosing differences in environments, and more time shipping new functionality for users.

Quali sono i vantaggi?

- ★ **Deployment simplicity** containers allow your application as a whole to be packaged, abstracting away the operating system, the machine, and even the code itself, so development and deployment are easier because containers are able to run virtually anywhere (Linux, Windows, and Mac operating systems; virtual machines or bare metal; developer's machine or data centers on-premises; public cloud).
- ★ **Easy portability** Docker image format for containers further helps with portability. Docker V2 image manifest is a specification for container images that allows multi-architecture images and supports content-addressable images

Quali sono i vantaggi?

★ **Operational efficiency and reliability** Containers are

perfect for Service Oriented Architectures/Applications because each service limited to specific resources can be containerized. Separate services can be considered as black boxes.

- This arises efficiency because each container can be health checked and started/stopped when needed independently from others
- Reliability arises because separation and division of labor allows each service to continue running even if others are failing, keeping the application as a whole more reliable

Quali sono i vantaggi?

- ★ **Easy Versioning** A new container can be packaged for each new application version including all needed dependencies, modules and libraries at the “right” version
- ★ **Security** Containers add an additional layer of security since the applications aren't running directly on the host operating system. There are security constraint if application running inside containers have root privileges

Problemi con più versioni

★ Java example

- /usr/lib/jvm/java-1.7.0-openjdk
 - /usr/lib/jvm/java-1.8.0-openjdk
- => /etc/alternatives/java_sdk -> /usr/lib/jvm/java-1.7.0-openjdk

★ Python example

- /usr/lib64/python2.7
 - /usr/lib64/python3
- => virtualenv
- => conda and anaconda

Come funziona un container?

```
$ python -V
```

Python 2.7.12 :: Continuum Analytics, Inc.

- If you install a package SomePackage, it will install on python 2.7 path
- /usr/lib64/python2.7/site-packages/orca
- If later you will need to build a software with python3 and this software need an orca library, you will not be able to compile because orca library is installed for python2.7
- Moreover, this is not an easy problem to debug
- Other issue: unintentionally upgrade an application that shouldn't be upgraded breaking another application

Come funziona un container?

- ★ Python “Virtual Environments” allow Python packages to be installed in an isolated location for a particular application, rather than being installed globally.
- ★ Virtual environments have their own installation directories and they don't share libraries with other virtual environments.

Come funziona un container?

- ★ Dockerfile to create a container packaging a specific python version and the application

Formati di containers

- ★ There are many container formats available. Examples:
 - **Docker** is a popular, open-source container format that is supported on Google Cloud Platform and by Google Kubernetes Engine.
 - **Singularity**: Singularity containers can be used to package entire scientific workflows, software and libraries, and even data
- ==> Singularity software can import your Docker images without having Docker installed or being a superuser

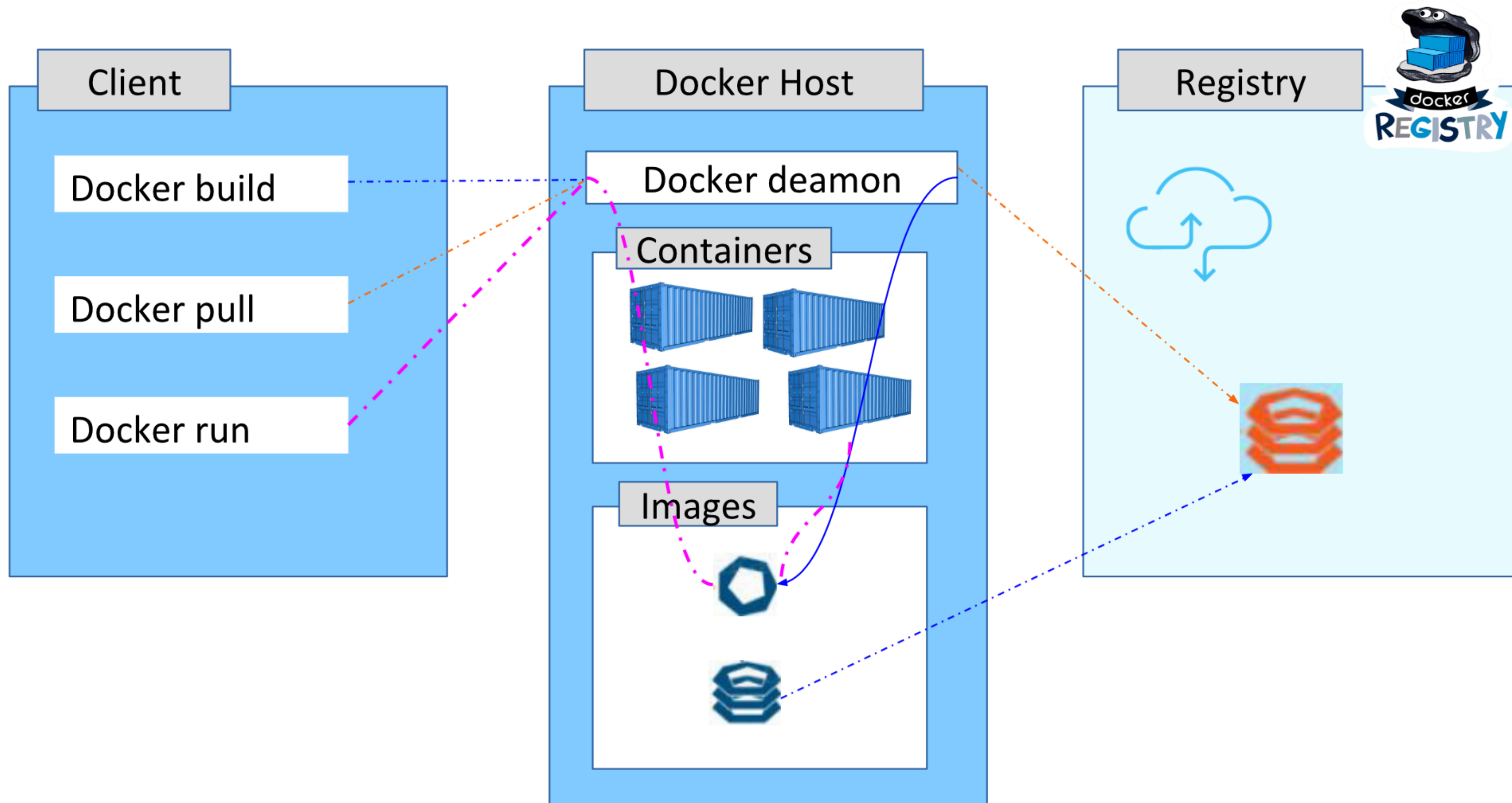
Docker Registry e Hub

★ **Docker Registry** is where the Docker Images are stored.

The Registry can be either a user's local repository or a public repository like a Docker Hub allowing multiple users to collaborate in building an application. Even with multiple teams within the same organization can exchange or share containers by uploading them to the Docker Hub.

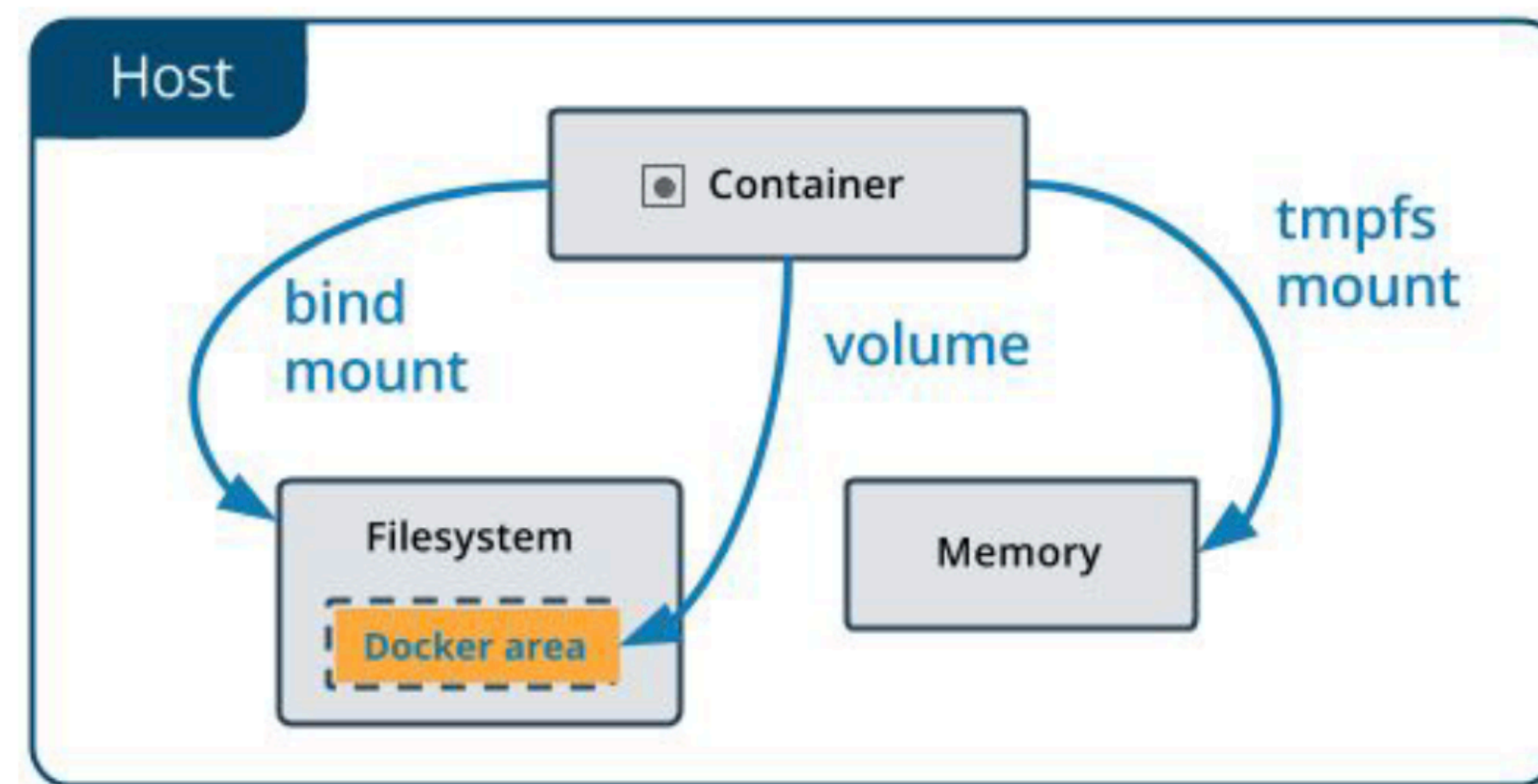
★ **Docker Hub** is Docker's very own cloud repository similar to GitHub.

Docker Registry e Hub



Docker Volume

- ★ Volumes are the preferred mechanism for persisting data generated by and used by Docker containers.
- ★ Volumes are completely managed by Docker.



```
docker run -d --name devtest -v myvol2:/app nginx:latest
```

Comandi

Install from [docker.com](https://docs.docker.com/get-docker/)

Simple commands:

- > `docker version`

- > `docker images`

Run a simple “hello world!” program:

- > `docker run hello-world`

Docker verifies if it is a local image with this name and if so, it runs it, elsewhere it tries to automatically download it:

Comandi

Name	Description
<code>attach</code>	Attach local standard input, output, and error streams to a running container
<code>build</code>	Build an image from a Dockerfile
<code>builder</code>	Manage builds
<code>checkpoint</code>	Manage checkpoints
<code>commit</code>	Create a new image from a container's changes
<code>config</code>	Manage Swarm configs
<code>container</code>	Manage containers
<code>context</code>	Manage contexts
<code>cp</code>	Copy files/folders between a container and the local filesystem
<code>create</code>	Create a new container
<code>diff</code>	Inspect changes to files or directories on a container's filesystem
<code>events</code>	Get real time events from the server

Comandi

<code>exec</code>	Execute a command in a running container
<code>export</code>	Export a container's filesystem as a tar archive
<code>history</code>	Show the history of an image
<code>image</code>	Manage images
<code>images</code>	List images
<code>import</code>	Import the contents from a tarball to create a filesystem image
<code>info</code>	Display system-wide information
<code>inspect</code>	Return low-level information on Docker objects
<code>kill</code>	Kill one or more running containers
<code>load</code>	Load an image from a tar archive or STDIN
<code>login</code>	Log in to a registry
<code>logout</code>	Log out from a registry
<code>logs</code>	Fetch the logs of a container
<code>manifest</code>	Manage Docker image manifests and manifest lists
<code>network</code>	Manage networks

Comandi

<code>node</code>	Manage Swarm nodes
<code>pause</code>	Pause all processes within one or more containers
<code>plugin</code>	Manage plugins
<code>port</code>	List port mappings or a specific mapping for the container
<code>ps</code>	List containers
<code>pull</code>	Download an image from a registry
<code>push</code>	Upload an image to a registry
<code>rename</code>	Rename a container
<code>restart</code>	Restart one or more containers
<code>rm</code>	Remove one or more containers
<code>rmi</code>	Remove one or more images
<code>run</code>	Create and run a new container from an image
<code>save</code>	Save one or more images to a tar archive (streamed to STDOUT by default)
<code>search</code>	Search Docker Hub for images

Comandi

<code>start</code>	Start one or more stopped containers
<code>stats</code>	Display a live stream of container(s) resource usage statistics
<code>stop</code>	Stop one or more running containers
<code>swarm</code>	Manage Swarm
<code>system</code>	Manage Docker
<code>tag</code>	Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
<code>top</code>	Display the running processes of a container
<code>trust</code>	Manage trust on Docker images
<code>unpause</code>	Unpause all processes within one or more containers
<code>update</code>	Update configuration of one or more containers
<code>version</code>	Show the Docker version information
<code>volume</code>	Manage volumes
<code>wait</code>	Block until one or more containers stop, then print their exit codes

Comandi

There are many publicly available images that we can use to work with Docker.
The example below pulls a hello-world image using the **docker pull** command:

```
[ ~ ] docker pull hello-world
```

```
Using default tag: latest
```

```
latest: Pulling from library/hello-world
```

```
0e03bdcc26d7: Pull complete
```

```
Digest: sha256:31b9c7d48790f0d8c50ab433d9c3b7e17666d6993084c002c2ff1ca09b96391d
```

```
Status: Downloaded newer image for hello-world:latest
```

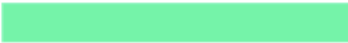
```
docker.io/library/hello-world:latest
```

```
[ ~ ] docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	bf756fb1ae65	12 months ago	13.3kB

Comandi

Create A Container



To create a container from an image we can use the **docker create** command

```
[ ~ ] docker create hello-world
```

```
2ffd5f2c5a7562fbf1d7b89a14c11a52e5843dd7938f380a8cd53f3952da99de
```

Comandi

To run a container we can use the **docker container start** command to start a container. The **-i** runs the container interactively and allows us to see the output

```
[ ~ ] docker container start -i 2ffd5f2c5a7562fbf1d7...
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

Comandi

To see what images are already installed on your machine you can use the **docker image ls** command. We can see our hello-world image below:

```
[ ~ ] docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu	latest	f63181f19b2f	13 hours ago	72.9MB
hello-world	latest	bf756fb1ae65	12 months ago	13.3kB

Comandi

To list the containers that we have built, we can use the **docker container ls** command. The **-a** flag allows us to see both stopped and running containers. There are two containers below, one that was built with the **docker create** command and the other that was built with **docker run**:

```
[ ~ ] docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5017fd2b94c2	hello-world	"/hello"	7 minutes ago	Exited (0) 7 minutes ago		stoic_nobel
5f0cea57eacf	ubuntu	"bash"	10 minutes ago	Exited (127) 8 minutes ago		condescending_neumann
2ffd5f2c5a75	hello-world	"/hello"	14 minutes ago	Exited (0) 13 minutes ago		hungry_mclaren

Comandi

Running containers interactively allows you to run commands inside the container if it supports it. We can use the openjdk image that we used before:

```
[ ~ ] docker run -it openjdk
Unable to find image 'openjdk:latest' locally
latest: Pulling from library/openjdk
a73adebe9317: Pull complete
8b73bcd34cfe: Pull complete
1227243b28c4: Pull complete
Digest: sha256:7ada0d840136690ac1099ce3172fb02787bbed83462597e0e2c9472a0a63dea5
Status: Downloaded newer image for openjdk:latest
Jan 21, 2021 4:48:58 PM java.util.prefs.FileSystemPreferences$1 run
INFO: Created user preferences directory.
| Welcome to JShell -- Version 15.0.2
| For an introduction type: /help intro

jshell> System.out.println("hello world");
Hello world
```

This allows us to execute java commands line by line in a Java shell

Comandi

To see what containers are currently running, we can use the **docker ps** command. This is useful when you are running containers in the background.

```
[ ~ ]docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
55e4a7c3ddcc	openjdk	"jshell"	11 seconds ago	Up 10 seconds		affectionate_kowalevski

Comandi

```
[ ~ ] docker run -it ubuntu bash
```

```
Unable to find image 'ubuntu:latest' locally
```

```
latest: Pulling from library/ubuntu
```

```
83ee3a23efb7: Pull complete
```

```
db98fc6f11f0: Pull complete
```

```
f611acd52c6c: Pull complete
```

```
Digest:
```

```
sha256:703218c0465075f4425e58fac086e09e1de5c340b12976ab9eb8ad26615c3715
```

```
Status: Downloaded newer image for ubuntu:latest
```

```
root@5f0cea57eacf:/#
```

Comandi

```
[bertocco@firiell containers]$ docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
```

```
Trying to pull repository docker.io/library/hello-world ...
```

```
latest: Pulling from docker.io/library/hello-world
```

```
d1725b59e92d: Pull complete
```

```
Digest:
```

```
sha256:0add3ace90ecb4adbf7777e9aacf18357296e799f81cab9fde47
```

```
0971e499788
```

```
Status: Downloaded newer image for docker.io/hello-world:latest
```

Comandi

- > `docker run --help` (i=interactive t=terminal)
- > `docker run -it ubuntu` (run ubuntu, minimized version, also in windows shell)

Now we are in ubuntu shell. Try `ls` to check the filesystem

- > `docker run -it --name ubuntu my_ubuntu` (gives a name)
- > `docker ps` (shows available containers)
- > `docker attach my_ubuntu` (to enter, use also the container ID)
- > `exit` (exits and closes, verify `docker ps`)
- > `docker images` (shows all downloaded images)
- > `docker rmi (-f) hello-world`

Comandi

- Launch docker with mounted directory:

```
docker run \  
--mount type=bind,source=HOST_PATH,target=CONTAINER_PATH \  
-it IMAGE:TAG
```

--mount takes a local HOST_PATH from your real computer and makes it appear in the container at CONTAINER_PATH. Note that both paths must be **absolute**.

-it Runs the container as an interactive terminal

IMAGE is the Docker image to run, and TAG is the specific version of that image

- Leave Docker and stop the container with `exit`

Esempio

```
mkdir cits4407  
docker run \  
--mount  
type=bind,source="$ (pwd) "/cits4407,target=/home/stud/perm \  
-it cits4407:v0
```

- We are mounting the local directory "\$ (pwd) "/cits4407
- `pwd` gets the absolute path of the current directory
- This directory is mounted to `/home/stud/perm`
- After this command succeeds, we will be in a new shell running inside our container

Warning!

- Remember, Docker containers are stateless. This means that **when a container stops, everything inside it will be deleted forever.**
- To avoid losing your work, **make sure you save it in your mounted directory**

```
docker run \  
--mount  
type=bind,source="$ (pwd) "/cits4407,target=/home/stud/perm \  
-it cits4407:v0  
cd ~/perm # everything in perm is permanent!  
echo "This file is permanent" > safe.txt  
exit  
ls cits4407
```

Esempio

1. Choose a container software (Docker, LXC, ...) and install a simple Linux image. Try SSHing into it.

Esempio

1. Choose a container software (Docker, LXC, ...) and install a simple Linux image. Try SSHing into it.

```
docker pull ubuntu
```

```
sudo docker pull ubuntu:20.04
```

```
sudo docker run -ti --rm ubuntu /bin/bash
```

Esempio

1. Choose a container software (Docker, LXC, ...) and install a simple Linux image. Try SSHing into it.

Salva:

```
sudo docker ps
```

```
docker commit -p container_id new_container_name
```

```
sudo docker commit -p 524aa76baafb myubuntu
```

Esempio

1. Choose a container software (Docker, LXC, ...) and install a simple Linux image. Try SSHing into it.

Renderlo persistente:

```
sudo mkdir -p Docker_Share
```

```
sudo docker stop container_id
```

```
sudo docker run -ti --rm -v ~/Docker_Share:/data ubuntu /bin/bash
```

Esercizi

1. Search and download a prebuilt container image for a popular web server (nginx, apache, ...)

Creare un container da Git



Search or jump to...

Pull requests

Issues

Marketplace

Explore



+



 sagars729 / TestRepo

Unwatch

1

Star

0

Fork

0

<> Code

! Issues

🔗 Pull requests

▶ Actions

📁 Projects

📖 Wiki

🛡 Security

📈 Insights

⚙ Settings

🔗 main

🔗 1 branch

🏷 0 tags

Go to file

Add file

Code

 sagars729 Update Workflow

3d3d77c 5 days ago 7 commits

	.github/workflows	Update Workflow	5 days ago
	.gitignore	Add JUnit Tests	5 days ago
	Calculator.java	Add Subtract Method	5 days ago
	README.md	first commit	5 days ago
	TestAdd.java	Add JUnit Tests	5 days ago
	TestSub.java	Add JUnit Tests	5 days ago
	junit-4.10.jar	Add JUnit Tests	5 days ago

README.md

TestRepo

About

No description, website, or topics provided.

Readme

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

Java 100.0%

© 2021 GitHub, Inc.

Terms

Privacy

Security

Status

Help



Contact GitHub

Pricing

API

Training

Blog

About

Creare un container da Git

Docker can build images automatically by reading the instructions from a Dockerfile. A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image.

The instruction is not case-sensitive. However, convention is for them to be UPPERCASE to distinguish them from arguments more easily.

```
FROM openjdk

RUN useradd -ms /bin/bash ojdk

RUN mkdir -p /home/ojdk/app/ && chown -R ojdk:ojdk /home/ojdk/app

WORKDIR /home/ojdk/app

COPY *.java ./
COPY junit-* ./

USER ojdk

RUN javac -cp "junit-4.10.jar:." *.java

COPY --chown=ojdk:ojdk . .

CMD [ "java", "-cp", "junit-4.10.jar:.",
      "org.junit.runner.JUnitCore", "TestAdd", "TestSub"]
```

Dockerfile

Creare un container da Git

Dockerfile Syntax

- **From** - The base image to use
- **Run** - Runs commands when building the docker image
- **Workdir** - Specifies the directory that commands are run from
- **User** - Switches users
- **Copy** - Copies Files
- **CMD** - Runs commands when running the container

```
FROM openjdk

RUN useradd -ms /bin/bash ojdk

RUN mkdir -p /home/ojdk/app/ && chown -R ojdk:ojdk /home/ojdk/app

WORKDIR /home/ojdk/app

COPY *.java ./
COPY junit-* ./

USER ojdk

RUN javac -cp "junit-4.10.jar:." *.java

COPY --chown=ojdk:ojdk . .

CMD [ "java", "-cp", "junit-4.10.jar:.", "org.junit.runner.JUnitCore", "TestAdd", "TestSub"]
```

Dockerfile

Creare un container da Git

Dockerfile Explained

1. Use the OpenJDK image to have a pre-configured java environment
2. Add a new user “**ojdk**” that we will be using for executing scripts
3. Create a directory that will contain our files and give permission to our user
4. Change the working directory to the directory we created
5. Copy the java and junit files
6. Switch to user ojdk
7. Compile all of our code
8. Copy files to the working directory and give permissions to ojdk
9. Run all tests

```
FROM openjdk

RUN useradd -ms /bin/bash ojdk

RUN mkdir -p /home/ojdk/app/ && chown -R ojdk:ojdk /home/ojdk/app

WORKDIR /home/ojdk/app

COPY *.java ./
COPY junit-* ./

USER ojdk

RUN javac -cp "junit-4.10.jar:." *.java

COPY --chown=ojdk:ojdk . .

CMD [ "java", "-cp", "junit-4.10.jar:.", "org.junit.runner.JUnitCore", "TestAdd", "TestSub"]
```

Dockerfile

Creare un container da Git

To build a docker image using a Dockerfile we can use the **docker image build** command and provide it the directory where the Dockerfile exists. The **--tag** option allows us to name and tag the docker image.

```
[TestRepo $ docker image build . --tag "calculator:latest"
Sending build context to Docker daemon 590.3kB
Step 1/9 : FROM openjdk
---> e105e26a0a75
Step 9/10 : COPY --chown=ojdk:ojdk . .
---> 47cff2b55e3c
Step 10/10 : CMD [ "java", "-cp", "junit-4.10.jar:.", "org.junit.runner.JUnitCore", "TestAdd", "TestSub" ]
---> Running in c8395bc770b4
Removing intermediate container c8395bc770b4
---> 6b345c94e511
Successfully built 6b345c94e511
Successfully tagged calculator:latest
```

Creare un container da Git

We can use **docker run -it** to run our image interactively and open a bash shell in our working directory:

```
[TestRepo $ docker run -it calculator bash  
[ojdk@419a727a1ca1 app]$ ls  
Calculator.class  README.md      TestSub.class  
Calculator.java   TestAdd.class  TestSub.java  
Dockerfile       TestAdd.java   junit-4.10.jar  
[ojdk@419a727a1ca1 app]$
```

An example Dockerfile

```
FROM ubuntu:22.04
```

```
RUN echo 'APT::Install-Suggests "0";' >> /etc/apt/apt.conf.d/00-docker
```

```
RUN echo 'APT::Install-Recommends "0";' >> /etc/apt/apt.conf.d/00-docker
```

```
RUN DEBIAN_FRONTEND=noninteractive \
```

```
apt-get update \
```

```
&& apt-get install -y python3 \
```

```
&& rm -rf /var/lib/apt/lists/*
```

```
RUN useradd -ms /bin/bash apprunner
```

```
USER apprunner
```

```
docker build . -t myubuntu
```



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

